

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

РАСЧЁТНО-ГРАФИЧЕСКАЯ РАБОТА

по дисциплине

«Базы данных»

**ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ СИСТЕМЫ УПРАВЛЕНИЯ И
МОНИТОРИНГА ТЕЛЕКОММУНИКАЦИОННОЙ СЕТИ**

Студент: *Группа № ИКС-433*

Г.В. Хоробров

В.А. Попов

Руководитель:

Старший преподаватель

К.И. Брагин

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ	4
1.1 Обзор предметной области	4
1.2 Функциональные задачи системы	4
2 ЛОГИЧЕСКОЕ ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ.....	5
2.1 Выделение сущностей и связей	5
2.2 ER-диаграмма	5
3 ФИЗИЧЕСКАЯ РЕАЛИЗАЦИЯ БАЗЫ ДАННЫХ.....	7
3.1 Инициализация структуры	7
3.2 Ограничения целостности (Constraints).....	7
3.3 Стратегия индексации.....	7
4 ФУНКЦИОНАЛЬНЫЕ ВОЗМОЖНОСТИ СИСТЕМЫ	9
4.1 Аналитические SQL-запросы	9
4.2 Программные объекты базы данных (Функции и триггеры)	10
4.2.1 Автоматическое создание аварий (Alarms)	10
4.2.2 Валидация закрытия аварий	10
4.2.3 Функции-утилиты	11
ЗАКЛЮЧЕНИЕ	12

ВВЕДЕНИЕ

В современных условиях телекоммуникационные сети становятся все более сложными и распределенными. Эффективное управление такой инфраструктурой невозможно без специализированных систем управления и мониторинга (Network Management Systems, NMS). Центральным компонентом любой NMS является база данных, обеспечивающая надежное хранение информации об оборудовании, его состоянии, метриках производительности и аварийных событиях.

Целью данной расчетно-графической работы является проектирование и реализация реляционной базы данных для системы управления и мониторинга телекоммуникационной сети. Разработанная система должна обеспечивать решение следующих задач:

1. Учет сетевого оборудования и его характеристик;
2. Мониторинг состояния физических и логических интерфейсов;
3. Сбор и хранение эксплуатационных метрик в реальном времени;
4. Регистрация системных событий и автоматическое формирование аварийных уведомлений;
5. Предоставление аналитических данных для оптимизации работы сети.

В ходе работы была спроектирована логическая модель данных, реализована физическая схема в СУБД PostgreSQL, настроены механизмы обеспечения целостности, а также разработаны SQL-запросы, хранимые процедуры и триггеры для автоматизации бизнес-логики.

Отчет состоит из четырех глав. В первой главе рассматривается описание предметной области и ее функциональные задачи. Вторая глава посвящена логическому проектированию базы данных. В третьей главе приводится описание физической реализации в PostgreSQL, включая ограничения целостности и индексы. Четвертая глава содержит программные объекты: аналитические запросы, функции и триггеры. В заключении подводятся итоги проделанной работы.

1 ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор предметной области

В качестве предметной области выбрана «Система управления и мониторинга телекоммуникационной сети» (Network Management System, NMS). Основная задача такой системы заключается в обеспечении бесперебойной работы сетевой инфраструктуры оператора связи или крупного предприятия.

Разрабатываемая база данных предназначена для централизованного учета сетевой инфраструктуры, сбора эксплуатационных метрик, регистрации событий и обработки аварийных ситуаций.

1.2 Функциональные задачи системы

Спроектированная база данных позволяет автоматизировать следующие функциональные задачи:

- **Учёт оборудования:** ведение реестра сетевых элементов (маршрутизаторы, коммутаторы, базовые станции), их производителей и типов устройств.
- **Учёт портов и интерфейсов:** хранение информации о физических и логических интерфейсах устройств, их статусе и пропускной способности.
- **Сбор метрик:** накопление временных рядов по ключевым показателям производительности (загрузка процессора, объем трафика на интерфейсе) с привязкой к временной метке.
- **Регистрация событий:** протоколирование системных событий (изменения конфигурации, обнаружение ошибок) с классификацией и уровнем критичности.
- **Управление авариями (Alarms):** отслеживание критических событий и управление инцидентами, контроль статуса их разрешения.
- **Аналитика и отчётность:** формирование агрегированных отчетов по загрузке оборудования, статистике инцидентов и активности в сети за заданные периоды.

2 ЛОГИЧЕСКОЕ ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ

2.1 Выделение сущностей и связей

На основе анализа предметной области были выделены следующие ключевые сущности:

- **manufacturer** (Производитель) --- содержит данные о компаниях-производителях оборудования (название, email).
- **network_elements_type** (Тип устройства) --- справочник типов сетевых элементов.
- **network_elements** (Сетевые элементы) --- основная сущность, хранящая информацию об устройствах (название, IP-адрес, текущий статус). Связана с производителем и типом.
- **interfaces** (Интерфейсы) --- порты сетевых устройств. Зависят от конкретного устройства.
- **metrics** (Метрики) --- телеметрические данные (нагрузка CPU, трафик), собранные с интерфейсов в определенные моменты времени.
- **event_types** (Типы событий) --- справочник категорий событий с указанием уровня их критичности.
- **events** (События) --- лог событий, произошедших на сетевых элементах.
- **alarms** (Аварии) --- активные инциденты, сгенерированные на основе критических событий, требующие реакции администратора.

2.2 ER-диаграмма

Логическая схема базы данных представлена на рисунке 1. Схема спроектирована с учетом требований третьей нормальной формы (3НФ), что позволяет избежать избыточности и аномалий обновления данных.



Рисунок 1 - ER-диаграмма базы данных системы мониторинга NMS

3 ФИЗИЧЕСКАЯ РЕАЛИЗАЦИЯ БАЗЫ ДАННЫХ

3.1 Инициализация структуры

Физическая схема базы данных реализована в СУБД PostgreSQL. Для создания структуры таблиц используется SQL-скрипт инициализации. Каждая таблица имеет первичный суррогатный ключ (типа SERIAL), а связи между таблицами реализованы через внешние ключи (FOREIGN KEY).

3.2 Ограничения целостности (Constraints)

Для обеспечения качества и корректности хранимых данных в схеме БД реализован ряд ограничений целостности:

- **PRIMARY KEY**: применяется во всех таблицах для уникальной идентификации каждой записи.
- **FOREIGN KEY**: обеспечивает ссылочную целостность между связанными сущностями (например, `network_element_id` в `interfaces` ссылается на `id` в `network_elements`).
- **NOT NULL**: защищает обязательные атрибуты от неполных записей (имя устройства, метрики, идентификаторы).
- **UNIQUE**: предотвращает дублирование данных. Например, уникальные IP-адреса устройств (`uq_ne_ip`), названия производителей, а также пары <<устройство + имя интерфейса>>.
- **CHECK**: контролирует бизнес-логику на уровне БД. Введены ограничения на допустимые статусы устройств (`active`, `down`, `maintenance`) статусы интерфейсов (`up`, `down`), уровни критичности событий и временные интервалы (время решения аварии `resolved_at` должно быть больше времени создания `created_at`).

3.3 Стратегия индексации

Для оптимизации производительности аналитических запросов и выборок созданы специализированные индексы:

- `idx_events_timestamp` по полю `events(timestamp)` --- ускоряет выборки событий за заданный временной период.
- `idx_alarms_ne` по полю `alarms(network_element_id)` --- обеспечивает быстрый поиск активных аварий по конкретному сетевому устройству.
- `idx_alarms_created` по полю `alarms(created_at)` --- ускоряет сортировку и фильтрацию инцидентов по дате возникновения.
- `idx_metrics_iface_time` (компози́тный индекс) по полям `metrics(interface_id, timestamp)` --- критически важен для аналитики временных рядов метрик конкретного интерфейса.
- `idx_ne_status` по полю `network_elements(status)` --- ускоряет поиск проблемных или отключенных устройств.

4 ФУНКЦИОНАЛЬНЫЕ ВОЗМОЖНОСТИ СИСТЕМЫ

4.1 Аналитические SQL-запросы

Для демонстрации возможностей спроектированной базы данных разработан комплекс SQL-запросов, решающий типичные задачи мониторинга. Ниже представлены наиболее значимые из них.

1. Выборка устройств с типом и производителем (JOIN):

```
SELECT ne.name, ne.ip, t.name AS type, m.name AS manufacturer
FROM network_elements ne
JOIN network_elements_type t ON ne.network_element_type_id = t.id
JOIN manufacturer m ON ne.manufacturer_id = m.id;
```

2. Агрегация количества событий по типам (GROUP BY):

```
SELECT et.name, COUNT(*) AS event_count
FROM events e
JOIN event_types et ON e.event_type_id = et.id
GROUP BY et.name;
```

3. Фильтрация устройств с высокой частотой событий за 30 дней (HAVING):

```
SELECT ne.name, ne.ip, COUNT(e.id) AS event_count
FROM network_elements ne
JOIN events e ON ne.id = e.network_element_id
WHERE e.timestamp > NOW() - INTERVAL '30 days'
GROUP BY ne.id, ne.name, ne.ip
HAVING COUNT(e.id) > 5;
```

4. Подсчет среднего количества интерфейсов по производителям (WITH/CTE):

```
WITH ne_iface_count AS (
    SELECT ne.id AS ne_id, ne.manufacturer_id, COUNT(i.id) AS i_count
    FROM network_elements ne
    LEFT JOIN interfaces i ON ne.id = i.network_element_id
    GROUP BY ne.id, ne.manufacturer_id
)
```

```

        GROUP BY ne.id, ne.manufacturer_id
    )
    SELECT m.name AS manufacturer, ROUND(AVG(nic.iface_count), 2)
    FROM ne_iface_count nic
    JOIN manufacturer m ON nic.manufacturer_id = m.id
    GROUP BY m.name;

```

5. Топ-3 устройства по объему трафика за сегодня (ORDER BY, LIMIT):

```

SELECT ne.name AS device_name, ne.ip, SUM(m.metric_value) AS total_traffic
FROM network_elements ne
JOIN interfaces i ON ne.id = i.network_element_id
JOIN metrics m ON i.id = m.interface_id
WHERE m.metric_name = 'traffic' AND m.timestamp >= CURRENT_DATE
GROUP BY ne.id, ne.name, ne.ip
ORDER BY total_traffic DESC
LIMIT 3;

```

4.2 Программные объекты базы данных (Функции и триггеры)

Для автоматизации бизнес-логики и поддержания целостности реализованы хранимые процедуры и триггеры.

4.2.1 Автоматическое создание аварий (Alarms)

Триггер `trg_event_create_alarm` срабатывает после вставки (AFTER INSERT) новой записи в таблицу `events`. Он вызывает функцию `fn_create_alarm`, которая проверяет флаг `is_alarm`. Если событие признано аварийным, система автоматически генерирует новую запись в таблице `alarms` с соответствующим уровнем критичности (`severity`), унаследованным от типа события.

4.2.2 Валидация закрытия аварий

Триггер `trg_alarm_validate_resolution` выполняется перед обновлением (BEFORE UPDATE) таблицы `alarms`. Связанная функция `fn_validate_alarm`

гарантирует, что время устранения аварии (`resolved_at`) строго больше времени ее возникновения (`created_at`). В случае нарушения логики генерируется исключение (`RAISE EXCEPTION`).

4.2.3 Функции-утилиты

Дополнительно реализованы функции для аналитики:

- `fn_get_device_stats(p_device_id)` --- возвращает сводную статистику по конкретному устройству (количество интерфейсов, событий, активных аварий и среднюю загрузку процессора).
- `fn_get_alarm_duration(p_alarm_id)` --- вычисляет продолжительность аварии (разницу между временем создания и закрытия).

ЗАКЛЮЧЕНИЕ

В результате выполнения расчетно-графической работы была успешно спроектирована и реализована реляционная база данных для системы управления и мониторинга телекоммуникационной сети (NMS). База данных способна решать задачи учета сетевой инфраструктуры, сбора эксплуатационных метрик и обработки аварийных событий.

В ходе выполнения работы были достигнуты следующие результаты:

1. Проведен анализ предметной области и выделены ключевые сущности, необходимые для мониторинга сети (устройства, интерфейсы, метрики, события, аварии).
2. Построена инфологическая и логическая схема данных (ER-модель) с соблюдением правил нормализации (3НФ).
3. Разработан SQL-скрипт инициализации для СУБД PostgreSQL, включающий создание таблиц, связей (FOREIGN KEY) и строгих ограничений целостности (CHECK, UNIQUE, NOT NULL).
4. Спроектирована и реализована оптимальная стратегия индексации, повышающая скорость работы аналитических запросов по временным рядам и статусам оборудования.
5. Написан комплекс из 10 функциональных SQL-запросов, демонстрирующих агрегацию данных, объединение таблиц, работу с обобщенными табличными выражениями (CTE) и фильтрацию.
6. Разработаны хранимые функции и триггеры, обеспечивающие автоматизацию бизнес-логики: автоматическое формирование аварий на основе критических событий и валидацию временных рамок при разрешении инцидентов.

Разработанная система полностью соответствует требованиям к продвинутому уровню выполнения задания и демонстрирует возможности СУБД PostgreSQL по надежному хранению и эффективной обработке телекоммуникационных данных. Данная база данных может служить надежным backend-ядром для разработки графического интерфейса системы NMS.